

Module Title:	Algorithms and Constructs (HDip in Computing - Feb 2025 cohort)
Assessment Title:	CA-2 (60%)
Lecturer Name:	Taufique Ahmed
Student Full Name:	Walter Barcellos Pereira Neto
Student Number:	2025213
Assessment Due Date:	Saturday, 29 November 2025, 11:59 PM
Date of Submission:	28/11/2025

Declaration

By submitting this assessment, I confirm that I have read the CCT policy on Academic Misconduct and understand the implications of submitting work that is not my own or does not appropriately reference material taken from a third party or other source. I declare it to be my own work and that all material from third parties has been appropriately referenced. I further confirm that this work has not previously been submitted for assessment by myself or someone else in CCT College Dublin or any other higher education institution.

Sorting Algorithm Choice - Recursive Algorithm	3
Searching Algorithm Choice	3
GitHub Repository Link	4

Sorting Algorithm Choice - Recursive Algorithm

For the sorting algorithm used in this project, I decided to choose Merge Sort walk together with recursive sorting method, and also because it offers a reliable level of performance when dealing with different numbers of input. Since the list of applicants comes from an external text file, the number of names can vary a lot, and I was looking for an approach that behaves consistently even if the list is large or not organised. Merge Sort is known for maintaining good efficiency almost every scenario, which made it a safe decision for this system.

Another reason for selecting Merge Sort is that it is stable and does not depend on the initial order of the data to do a good job. Some other algorithms may show variations in performance depending on how the input is arranged. Because of this, I wanted to avoid methods that could work quickly in some cases but then become slower in others. The premise here was reliability and consistent performance.

To conclude, Merge Sort also gave me a clear structure to integrate with the rest of the system. The sorting result is used again during the searching phase, so I wanted a sorting algorithm that guarantees the list will be organised in a stable and predictable order. Considering all of this, Merge Sort felt like the most balanced and sensible choice for the task.

Searching Algorithm Choice

For searching inside the applicants list, I used Binary Search, because it takes advantage of the fact that the data is already sorted by the merge recursive sort. Once the list is ordered, Binary Search turns out to be one very efficient to locate a specific name, and that fits with the idea of keeping the system fast and responsive even if the number of applicants grows.

Compared to Linear Search, which checks every item one by one, Binary Search reduces the number of comparisons alot. Since the premise was to make it efficient, it didn't seem a good choice to use a slower search technique when the data was already prepared for a more suitable one. With Binary Search, We can quickly determine if a name exists or not, and the behaviour remains consistent regardless of the size of the list.

Another factor that influenced the choice was the clarity of the method from a user-perspective. When running the program, the search operation feels immediate, which improves the user experience and supports the idea of building a system that is functional and well-structured. Because the sorted list from Merge Sort aligns perfectly together with Binary Search, both algorithms complement each other well inside the project.

GitHub Repository Link

https://github.com/WalterBPN/CA_2